PromptABI: Static Verification for the Discrete Interface Layer of LLM Systems

PromptABI Contributors

June 4, 2026

Abstract

Large language model applications increasingly fail at a layer that is neither neural nor traditionally tested: the discrete interface layer made of chat templates, tokenizer metadata, special tokens, stop policies, structured output schemas, tool definitions, provider request envelopes, and framework truncation rules. PromptABI is a static and differential verifier for that layer. Its core claim is deliberately narrow and therefore practical: before loading model weights or running inference, a developer can prove that important prompt and protocol states are possible, impossible, ambiguous, or unsafe under the exact artifacts that will be used in production.

This paper presents the repository as it now stands and the research system it is designed to become. The implementation already contains a typed Python package, a deterministic command line workflow, a public embedding API, a typed artifact model, a stable diagnostic contract with text, JSON, and SARIF renderers, offline version-pinned artifact loading, a tokenizer abstraction and differential harness validated against real Hugging Face `tokenizers`, `tiktoken`, and SentencePiece libraries, deterministic finite automata, finite-state transducers with composition and projection, a finite-contract solver, a minimal verification session, focused tests, examples, a fixture corpus layout, a CPU-only benchmark, documentation, and contribution guidance. The remaining checkers fit into that surface rather than requiring a rewrite. We describe the transducer-oriented formalism, the artifact model, the diagnostic contract, the first benchmark line, and the planned evaluation against real tokenizer, template, structured-output, provider, and budget failures. The result is a path toward a tool that is useful in CI and credible as a systems paper: it catches failures that arise before any model logit is relevant.

1 Motivation

Modern LLM applications are built from small, fragile protocol objects. A chat template decides which bytes separate roles. A tokenizer decides whether those bytes are ordinary content or special tokens. A stop policy decides where output is truncated. A tool schema decides which JSON fragment is valid. A provider adapter decides whether a tool call appears as a message, a delta, a function object, or a model-specific sentinel. A framework budget policy decides which messages, retrieved chunks, and tool definitions survive when the prompt exceeds the context window.

These objects compose into a pipeline:

```
messages -> chat template -> byte/string prompt -> tokenizer -> token stream
        -> constrained decoder / stop logic / tool parser -> application parser
```

The failure modes are concrete. A user field can contain a string that renders as an assistant delimiter. A stop string can fire inside a valid JSON string, making the application parse a malformed prefix. A tokenizer update can change special-token IDs while tests still pass because they never inspect the token stream. A RAG prompt can preserve the latest user message while silently dropping the safety instruction or the tool definition that gives the message meaning. These are not model-behavior questions. They are contract questions.

PromptABI treats those contracts as first-class verification targets. It does not promise semantic prompt-injection resistance. It does not ask whether a model will follow an instruction. It asks whether the interface representation admits a structural state that the developer did not intend. That boundary makes the tool useful without GPUs, private model access, or nondeterministic generation.

2 Current Repository Artifact

The current repository milestone establishes the skeleton needed for serious verification work. It uses a Python `src/` layout, declares package metadata in `pyproject.toml`, exposes the `promptabi` console entrypoint, ships `py.typed`, and includes focused tests for the public API, artifact model, diagnostic contract, configuration loader, and CLI behavior. The implementation is intentionally small, but it is not a paper mockup: it loads real JSON configuration, discovers configs from subdirectories, accepts explicit artifact overrides, normalizes artifact paths, builds typed artifact bundles, materializes local and metadata-only remote artifacts, validates sha256 pins, adapts real tokenizer backends into stable encode/decode metadata, extracts automata, transducer, and solver witnesses for finite formal contracts, prepares a cache directory for future expensive products, emits deterministic typed diagnostics in text, JSON, and SARIF, returns stable policy-controlled process exit codes, and runs a CPU-only smoke benchmark against an example bundle.

The repository layout is designed to scale:

Path	**Purpose**
`src/promptabi`	Typed package, artifact model, formal automata, finite-state transducers, finite-contract core, offline loaders, CLI, configuration loading, diagnostic model, renderers, and verification session.
`tests`	Focused tests for package API and CLI determinism.
`examples/minimal`	A typed artifact config, messages file, tool schema, and answer schema used by tests and docs.
`fixtures`	Seed corpus layout for tokenizer and chat-template bundles with provenance metadata.
`benchmarks`	CPU-only benchmark entrypoints for repeatable performance baselines.
`docs`	MkDocs-ready documentation for architecture, concepts, and check families.

This milestone matters because later checkers must not accrete as one-off scripts. They need stable input objects, stable diagnostics, stable renderers, stable examples, and stable tests from the beginning. PromptABI now has that surface and a model that names each interface object explicitly.

3 Core Abstraction

The unifying abstraction is that LLM interface systems are compositions of finite-state transducers over byte, string, token, and structured alphabets, with partial parsers at the boundaries. A chat template maps structured messages to a string. A tokenizer maps strings to token streams and back, subject to special tokens, normalization, byte fallback, and added-token behavior. A constrained decoder or grammar describes a language of valid outputs. A stop policy cuts a prefix from that language. An application parser maps the resulting bytes back to an abstract syntax tree or rejects them.

Many high-value questions become language questions:

Property	**Question**
Reachability	Can a declared delimiter, stop sequence, or control token be produced under the selected tokenizer and grammar?
Over-reachability	Can the same sequence appear inside user-controlled content, a JSON string, a tool argument, or a markdown block?
Emptiness	Is the product of tokenizer assumptions and grammar constraints empty, so no valid output can be generated?
Non-forgeability	Can an attacker-controlled field render as role or provider structure after template expansion?
Must-survive	Does a required prompt segment remain after the framework truncation policy is applied?

The abstraction is not used to pretend arbitrary Python, arbitrary Jinja, and arbitrary provider behavior are finite-state. Instead, PromptABI should classify each check as sound, complete, bounded, heuristic, or abstaining. That taxonomy is crucial: it lets the tool be useful where it is precise and honest where it is not.

4 Artifact Model

PromptABI uses a typed artifact model because the same file can participate in multiple checks. A tokenizer config is needed for token-boundary analysis, special-token drift, chat-template parsing, and prompt-budget accounting. A tool definition is needed for role-boundary analysis, structured-output compatibility, provider migration checks, and budget survival. A diagnostic must therefore identify what artifact it refers to without coupling the renderer to a particular loader.

The public API now provides `ArtifactKind`, `ArtifactLocation`, `ArtifactProvenance`, `ArtifactBundle`, and typed artifacts for tokenizers, chat templates, special-token maps, stop policies, schemas, grammars, tool definitions, prompt segments, provider configs, and framework truncation configs. It also provides `ArtifactRef`, `SourceSpan`, `WitnessTrace`, `Diagnostic`, `VerificationConfig`, `VerificationSession`, and `VerificationResult`. These types establish the compatibility contract:

- artifacts have stable kinds, names, local paths or URIs, provenance, and optional versions, revisions, hashes, licenses, and sources;
- kind-specific fields capture stop strings, schema dialects, grammar types, tool names, prompt segment requirements, provider families, and truncation strategies;
- spans are one-based and validated at construction time;
- witnesses are explicit traces, not prose hidden in the message string;
- diagnostics have rule IDs, severities, artifacts, spans, witnesses, and suggestions;
- results serialize to deterministic dictionaries for JSON and snapshots.

The config loader exercises the model by accepting both legacy path strings and typed artifact objects, resolving local paths relative to the config file, and retaining URI-backed artifacts without subjecting them to local existence checks. This small behavior prevents a future class of flaky tests and confusing CLI output: diagnostics refer to the same canonical artifact no matter where the command is launched from.

5 Formal Verification Core

PromptABI now includes the first executable formal core rather than only a roadmap for one. The core has two deliberately finite pieces. The first is a deterministic finite automata library over explicit alphabets. It constructs literal languages, finite-language tries, and prefix-closed literals; supports completion, complement, union, intersection, and difference; checks emptiness; extracts shortest witnesses with state-based breadth-first search; and serializes automata products deterministically. This is enough to express early reachability facts such as whether a stop string intersects a grammar prefix, whether a user-controlled field can equal a role delimiter, and whether removing forbidden control strings leaves any accepted user language.

The second piece is a finite contract solver. A contract declares variables over Boolean, enum, integer-range, and bounded-string domains, then composes named constraints with equality, disequality, conjunction, disjunction, negation, implication, membership, string length, and substring containment. When `z3-solver` is available the adapter lowers those finite domains and constraints to Z3, including explicit domain restrictions for bounded strings. When Z3 is absent, the same contract runs through deterministic exhaustive finite-domain enumeration. Both backends return a stable result object with `sat`, `unsat`, or `unknown` status, backend provenance, counterexample assignments for satisfiable bad states, checked-assignment counts, and minimal enumerated unsat cores when no assignment exists.

The core now adds the missing relational layer: finite-state transducers over explicit input and output alphabets. A transducer transition carries an input label, an output label, or epsilon on either side. This directly models render-like insertions, tokenizer-like recodings, decode-like normalization, and provider-envelope rewrites. The implementation accepts finite relations, literal string mappings, identity relations, exact composition over a shared middle alphabet, input and output projection back to deterministic automata, shortest input/output witnesses with path labels, and a conservative over-approximation that pairs the independent input and output projections when only a relation envelope is justified. Composition preserves the exact/over- approximate provenance flag so later diagnostics can distinguish proof from bounded approximation.

This split is important for the research claims. Automata should prove language and reachability properties; transducers should model interface mappings and their compositions; SMT should prove finite symbolic compatibility over discrete pipeline facts such as roles, escaping flags, bounded content, token budgets, schema preconditions, and stop-policy states. The fallback enumerator is not a replacement for richer solving, but it makes the supported fragment executable, testable, and privacy-preserving even in minimal CI environments.

The current tests exercise the core against real code rather than sketches. They prove that automata products extract a shortest stop-string witness, that difference excludes a role delimiter while preserving safe user strings, that transducer composition reconstructs a render-then-tokenize witness, that projections recover accepted input and output languages through epsilon labels, that projection-based over-approximation intentionally admits independent pairs, that a finite contract emits a concrete role-forgery counterexample, that contradictory role constraints produce a minimal unsat core, that Boolean compositions are recognized as unsatisfiable, and that the installed Z3 backend returns the same kind of concrete assignment for a satisfiable finite contract. This is the first formal benchline on which role-boundary, stop-reachability, grammar-emptiness, and must-survive budget checkers can be built.

6 Version-Pinned Artifact Loading

Artifact identity is not enough for a verifier. A checker that proves a property about `tokenizer_config.json` must know which bytes, directory manifest, provider snapshot, or upstream revision it inspected. PromptABI therefore adds an offline loader layer that turns typed artifact references into deterministic `LoadedArtifact` summaries. The loader does not contact third-party services and does not require tokenizer, provider, or grammar libraries. It validates the reproducibility envelope first, so deeper semantic checkers inherit stable inputs.

The implemented loader covers the artifact sources that appear earliest in real LLM interface workflows:

Source	**Loaded summary**
Local files	Streamed sha256 digest, byte size, pin validation, and mismatch diagnostics.
Hugging Face refs	Offline repository ID, optional artifact path, revision extraction, and warnings for missing or movable revisions.
Tokenizer directories	Stable manifest over sorted relative file names, file sizes, and content hashes; requires tokenizer metadata.
GGUF stubs	Header-level GGUF magic, version, tensor count, metadata-key count, and streamed file hash without loading weights.
Provider snapshots	Deterministic JSON validation for provider name and captured request, response, or streaming shape metadata.
Fixture archives	Zip/tar member lists and archive-byte hashes without extraction or timestamp-dependent manifests.

Pinning is intentionally behavior-safe. A missing local file is an error because the verifier cannot inspect it. A malformed sha256 or mismatched sha256 is an error because the configured claim is contradicted. An otherwise readable but unpinned artifact is a warning by default, allowing existing projects to adopt strict reproducibility incrementally or fail on warnings in CI. In-memory API artifacts are treated as embedded objects rather than external supply-chain inputs.

This capability is more than bookkeeping. It is the first supply-chain boundary for the research system: every future role-boundary, stop-reachability, grammar-emptiness, provider-migration, and budget-survival finding can cite the exact artifact reference, actual hash, revision strength, and loader witness that made the claim reproducible.

7 Tokenizer Abstraction Layer

Tokenizers are not a peripheral convenience in PromptABI; they are one of the central semantic boundaries. A role delimiter, stop string, JSON escape, or tool sentinel is only meaningful after normalization, added-token recognition, special-token handling, byte fallback, tokenization, and decoding are known. The repository now exposes a backend-neutral tokenizer interface that records token IDs, token surfaces, byte spans, special-token flags, added-token flags, normalization steps, decoded text, and round-trip metadata.

The implemented layer covers four practical families:

Backend	**Capability**
Byte-level	Deterministic byte alphabet with greedy added-token matching, portable Unicode normalization rules, UTF-8 byte spans, and exact decode reconstruction.
Hugging Face `tokenizers`	Local `tokenizer.json` and tokenizer-directory loading, real backend encode/decode, backend normalizer metadata, offsets converted to byte spans, and added-token detection.
`tiktoken`	Installed encoding adapters such as `cl100k_base`, special-token handling, token-byte metadata, and round-trip checks against the real library.
SentencePiece	Local `.model` loading, piece/id extraction, decode comparison, and best-effort byte spans where the piece surface can be matched.

This abstraction is deliberately thinner than a tokenizer reimplementation. Its job is to expose the observable behavior of real libraries in a stable shape that formal checks can consume. A stop-reachability checker can ask for byte spans and token IDs without knowing whether the source was ByteLevel BPE, SentencePiece, or `tiktoken`. A role-boundary checker can see whether a delimiter is an ordinary byte sequence, an added token, or a special token. A drift checker can compare normalized text, token IDs, token surfaces, and decode results across versions.

The current tests validate this layer against real code rather than mocks. A trained local Hugging Face ByteLevel BPE tokenizer is compared against `tokenizers.Tokenizer.encode` and `decode`. The `tiktoken` adapter is compared against `cl100k_base`, including special-token and byte-fallback behavior. A temporary SentencePiece model is trained and compared against `SentencePieceProcessor`. The byte-level adapter separately proves normalization, added-token, special-token, UTF-8 span, and round-trip behavior. These tests form the first differential benchline for the eventual tokenizer corpus.

7.1 Tokenizer Differential Harness

PromptABI now has a reusable differential harness rather than one-off adapter assertions. Each case records the input text, whether the backend should inject special tokens, whether decoding should skip them, and an oracle expectation collected from the real library. The harness then compares PromptABI's stable abstraction against that oracle across token IDs, token surfaces, decoded text, normalized text, normalization steps, special-token IDs, added-token IDs, required byte spans, and normalized round-trip behavior. Failures are returned as deterministic mismatch records with stable dictionaries suitable for future snapshot tests and corpus reports.

The harness already caught a realistic edge case during development: Hugging Face stores special tokens in its added-token decoder, but downstream structural checks need to distinguish provider/model control tokens from ordinary added tokens such as tool sentinels. The adapter now treats specialness and added-token status as separate flags, so a role-boundary checker can reason about `<s>` and `</s>`

differently from an application-defined `<tool>` token. This is the sort of small tokenizer-library semantic difference that later formal checks must get right before their witnesses are credible.

8 Diagnostic Contract

PromptABI's diagnostic contract is designed for CI, code scanning, embeddings, and regression snapshots. A useful finding is not merely a line of text. It needs a stable rule ID, a severity, a precise artifact reference with provenance, a source span when available, a witness that reproduces the failure, a deterministic fingerprint, and a suggested remediation. If two releases emit different diagnostics for the same artifact bundle, that difference should be intentional, reviewable, and testable.

The repository now renders the same diagnostic object in human text, deterministic JSON, and SARIF 2.1.0. This matters early: once a verifier has many rules, unstable output becomes a tax on CI, pull-request review, and paper reproducibility. PromptABI therefore fixes ordering, key names, optional-field behavior, and code-scanning fingerprints before high-volume checkers are introduced. A future role-forgery finding can reuse the same shape:

```
{
  "rule_id": "role-boundary-nonforgeability",
  "severity": "error",
  "fingerprint": "stable-sha256-prefix",
  "message": "user.content can render an assistant delimiter",
  "artifact": {
    "kind": "chat-template",
    "name": "tokenizer_config",
    "revision": "pinned-upstream-revision",
    "sha256": "artifact-hash"
  },
  "span": {"path": "tokenizer_config.json", "start_line": 12, "start_column": 3},
  "witness": {
    "summary": "malicious user content reaches assistant header",
    "steps": [
      {"action": "render template", "input": "user.content", "output": "..."},
      {"action": "tokenize prompt"},
      {"action": "locate forged boundary"}
    ]
  },
  "suggestions": ["Escape user content before inserting role delimiters."]
}
```

The first implemented rule, `repository-skeleton`, is intentionally informational. It proves the machinery works without claiming a security or compatibility property that has not yet been implemented. That restraint is important for trust: checkers should only claim properties that the code has actually proved.

The SARIF renderer maps PromptABI severities to SARIF levels, emits a rule table, attaches artifact locations and source regions when available, and stores the PromptABI fingerprint in `partialFingerprints`. This makes the first CLI workflow immediately compatible with GitHub code scanning while keeping the diagnostic source of truth in Python types rather than renderer-specific structures.

9 CLI and Embedding API

The command line entrypoint is:

```
promptabi verify
promptabi verify --config examples/minimal/promptabi.json
promptabi verify --artifact schema=schemas/answer.json --fail-on warning
promptabi verify --config examples/minimal/promptabi.json --format json
promptabi verify --config examples/minimal/promptabi.json --format sarif
promptabi verify --cache-dir .cache/promptabi --verbose
```

In the current milestone the workflow validates repository wiring, local existence, reproducible pins, and loader compatibility. It discovers `promptabi.json` or `.promptabi.json` by walking upward from the current directory unless `-config` is explicit. It accepts repeated `-artifact NAME=PATH_OR_URI` overrides so CI can verify generated or matrix-selected artifacts without rewriting the repository config. It prepares a cache directory selected by `-cache-dir`, `PROMPTABI_CACHE_DIR`, `XDG_CACHE_HOME`, or a user cache fallback. It provides quiet and verbose text modes while keeping JSON and SARIF stable for machines.

The exit-code policy is configurable. The default fails only on error-level diagnostics. CI can instead fail on warnings, fail on any diagnostic, or never fail the process while still emitting machine-readable results. Malformed invocations and invalid configs return code 2, preserving the distinction between a broken workflow and a real artifact-contract finding.

The same behavior is available through the embedding API:

```
from promptabi import VerificationSession

result = VerificationSession.from_config_file("promptabi.json").run()
if not result.ok:
    raise SystemExit(1)
```

The point is not that the current checker is deep. The point is that all later checkers have an integration path. A LangChain plugin, a pre-commit hook, a GitHub Action, a notebook visualization, and a provider-fixture replay harness can all depend on the same session and result objects. That prevents the common research-tool failure mode where each experiment has its own incompatible input format and output convention. The first workflow milestone also establishes practical CI affordances—discovery, overrides, caches, verbosity, deterministic rendering, and explicit failure thresholds—before expensive formal checkers arrive.

10 Role-Boundary Non-Forgeability

The first major formal checker will analyze whether user, tool, RAG, or other attacker-controlled fields can render as structural prompt boundaries. In ChatML-like templates the relevant strings may be `<|im_start|>`, `<|im_end|>`, role names, or assistant prefixes. In Llama-style templates they may be header IDs and end-of-turn sentinels. In instruction-tag templates they may be `[INST]`, `[/INST]`, or custom fine-tune delimiters. In XML-like tool formats they may be tool-call start and end tags.

The checker should model the rendered prompt as regions: system, user, assistant, tool, developer, provider envelope, and generated assistant prefix. Then it should ask whether content controlled by one region can create syntax that is interpreted as another region. A witness should include a minimized malicious field, a rendered excerpt, the tokenized representation, and the exact boundary that becomes ambiguous or forged.

This property is structural, not semantic. PromptABI can prove that a template allows user content to create an apparent assistant turn. It cannot prove that a model will obey that turn. That distinction makes the result more useful, not less, because the finding remains valid across model weights and sampling parameters.

11 Stop and Grammar Reachability

Stop policies often look simple: if the generated text contains a string, cut there. In structured-output systems this simplicity is dangerous. A stop string can be unreachable because the tokenizer or grammar cannot produce it. It can be ambiguous because multiple token paths or decoded byte strings correspond to the same visible text. It can overreach because the string appears inside a valid JSON string, tool argument, markdown fence, or XML-like field before the structured object is complete.

PromptABI should analyze stop policies jointly with tokenizers and grammars. A useful witness for overreachability is not just "the stop appears." It is a valid output prefix, the parser state at truncation, the exact stop firing point, and the malformed or prematurely accepted structure received by the application parser. A useful witness for unreachability includes the stop sequence, its byte form, the relevant tokenizer segmentation, and the grammar state that prevents production.

This check family is attractive because it is both practical and formal. It catches flaky production bugs, and it can be stated as a reachability property over a product of tokenizer, grammar, stop automaton, and parser states.

12 Structured Output and Grammar Emptiness

Structured-output libraries increasingly compile JSON Schema, regex fragments, EBNF-like grammars, Pydantic models, or provider-specific tool schemas into a decoder constraint. The application then parses the resulting bytes with a different parser. Bugs appear when those languages do not match. A schema may be valid under a Python validator but compile into an empty grammar under a specific backend. A grammar may accept strings the application parser rejects. A tokenizer may expose Unicode, escaping, whitespace, or added-token behavior that creates ambiguous structured outputs.

PromptABI should normalize the supported JSON Schema subset into a grammar IR with source maps. It should compile that subset to automata or a backend-neutral grammar representation, then differentially compare behavior against real validators and structured-output libraries where possible. The key theorem shape is: under the supported fragment assumptions, if the product is empty, then no sequence accepted by the tokenizer and grammar backend can produce a valid application object.

Equally important is abstention. Arbitrary schemas with recursive references, custom validators, remote references, or backend-specific extensions should not be silently approximated as safe. The diagnostic should state exactly which fragment forced abstention and where that fragment appears.

13 Must-Survive Budget Verification

Token-budget failures are common because different layers count and truncate in different ways. A framework may drop old messages from the left. A RAG pipeline may trim chunks after adding metadata. A serving engine may reserve tokens for tools or generation prompts. A tokenizer mismatch may make a prompt fit in development but overflow in production. The result can be subtle: the user message survives, but the system instruction, output-format requirement, citation-bearing span, or tool schema disappears.

PromptABI should represent prompt components as named segments with must-survive constraints. A budget checker then combines segment token counts, special-token overhead, tool-definition overhead, generation reserve, and the selected framework truncation policy. The property is not "the prompt fits." The property is "all required segments survive under the policy that will be used."

A good diagnostic should show the segment table, truncation boundary, dropped fields, framework policy, tokenizer identity, and a minimized counterexample configuration. This makes the finding actionable: the developer can reduce retrieval size, reserve more budget, change the truncation policy, pin a tokenizer, or mark a different segment as required.

14 Differential Validation

PromptABI's abstractions must be checked against the libraries developers actually use. The verifier should not implement a tokenizer, chat-template engine, or grammar compiler and assume equivalence. It should differentially test its abstraction against Hugging Face tokenizers, `apply_chat_template`, `tiktoken`, SentencePiece where practical, llama.cpp metadata, vLLM and OpenAI-compatible request fixtures, and structured-output backends such as Outlines, xgrammar, llguidance, lm-format-enforcer, Guidance, and Instructor.

The testing strategy has three tiers:

- unit tests for public API and deterministic rendering;
- tokenizer abstraction tests against real Hugging Face `tokenizers`, `tiktoken`, SentencePiece, and byte-level adapters;
- loader tests for local files, Hugging Face refs, tokenizer directories, GGUF stubs, provider snapshots, and fixture archives;
- fixture tests for minimized artifact bundles with expected diagnostics;
- corpus tests for popular model and framework configurations pinned by revision.

Differential tests are especially valuable for version drift. A tokenizer or provider SDK update can change behavior without changing application code. PromptABI should make that drift visible as an artifact contract change rather than a mysterious downstream parsing failure.

15 Fixture Corpus

The fixture corpus is the bridge between research claims and developer trust. It should cover popular instruct-model templates, structured-output schemas, tool-call envelopes, provider fixtures, RAG budget cases, and known delimiter collision patterns. Each entry should include provenance, license notes, exact upstream revisions, hashes, minimized repro inputs, expected diagnostics, and unsupported-fragment notes.

The current repository contains the initial layout and a minimized ChatML-like tokenizer config. That seed is intentionally small, but it establishes the file shape expected by future corpus entries. A mature corpus will include Llama, Mistral, Qwen, Gemma, Phi, DeepSeek, Zephyr, OpenAI-style adapters, llama.cpp GGUF metadata, common fine-tune templates, real tool schemas from open-source agents, anonymized provider traces, and synthetic stress cases with known labels.

The corpus should avoid secrets and network requirements. Provider fixtures should be recorded request and response shapes, not live API calls. Model artifacts should be metadata and tokenizer files, not weights. This keeps CI fast, reproducible, and privacy-preserving.

16 Benchmark Lines

The first benchmark lines are deliberately modest: repeatedly load the minimal configuration, validate its pinned artifacts, run the skeleton verifier, and exercise the formal core through automata witness extraction and finite-contract solver tests. Their value is not in absolute runtime; they prove that benchmarks and focused tests are CPU-only, deterministic, executable from the repository, and able to fail if verification unexpectedly emits an error.

Future benchmark lines should measure:

Benchmark	**Measurement**
Tokenizer abstraction	Encode/decode metadata extraction, special-token handling, and normalization cost.
Template execution	Supported Jinja fragment rendering, symbolic execution, and abstention detection.
Automata and transducers	Completion, union, intersection, difference, emptiness, transducer composition, projection, over-approximation, and shortest witness extraction.
Finite-contract solving	Z3 and exhaustive finite-domain solving over booleans, enums, integer ranges, bounded strings, length, membership, and containment constraints.
Stop analysis	Reachability and overreachability across JSON, markdown, and tool-call regions.
Budget verification	Segment accounting and truncation simulation for framework policies.
Corpus verification	Cold and warm runs across pinned artifact bundles.

Benchmarks should report runtime, memory where practical, abstention rate, diagnostic count, and witness size. The paper evaluation can then separate precision, recall, agreement, and performance rather than mixing them into a single anecdotal demo.

17 Documentation and Developer Experience

A 1000-star verification repo needs more than algorithms. It needs a first-run experience where a developer immediately understands what failed and how to fix it. The README now leads with a concrete CLI command, a CPU-only explanation, and the three high-value check families. The docs tree explains architecture, quickstart, check families, and the artifact model. The contribution guide states the central norms: deterministic output, CPU-only tests, minimized fixtures, typed public APIs, and explicit soundness boundaries.

The examples directory is equally important. PromptABI should accumulate small applications that intentionally contain bugs: role-delimiter injection, stop overreach in JSON, RAG truncation that drops citations, provider migration that changes a tool-call AST, and tokenizer drift that changes special-token behavior. Each example should include both the failing artifact bundle and the fixed version. That gives users practical recipes and gives maintainers stable regression tests.

The CLI should remain terse by default and expandable on demand. A CI run needs deterministic diagnostics and exit codes. A local debugging session needs witnesses, rendered excerpts, token tables, and fix suggestions. A future `promptabi explain` command can bridge those modes.

18 Evaluation Design

The evaluation should answer four questions.

1. Does PromptABI find real interface bugs in popular LLM application stacks?
2. Are the findings precise enough that developers would act on them?
3. Does the verifier abstain honestly when artifacts exceed the supported fragment?
4. Do automata and SMT-backed finite contracts produce reproducible witnesses?
5. Is the runtime low enough for pull-request CI?

The experimental setup should include labeled corpora, differential agreement tests, mutation-based stress cases, and version-drift tests. Precision and recall can be measured on synthetic and real-bug corpora where ground truth is known. Differential agreement can be measured against tokenizer libraries, chat template rendering, schema validators, grammar backends, and provider fixture replay. Runtime can be reported for individual checks and corpus-wide runs. Witness quality can be evaluated by minimization size, reproducibility, and whether the witness includes enough artifact context to file an upstream bug.

The strongest paper result would be upstreamable bugs: minimized reports that maintainers of templates, adapters, structured-output libraries, or framework integrations accept as real. That evidence is more compelling than a large synthetic benchmark alone.

19 Threat Model and Non-Goals

PromptABI's threat model is structural. Attacker-controlled content may enter user messages, tool outputs, retrieval chunks, metadata, or provider-compatible fields. The verifier asks whether that content can be represented as control structure, whether output can be truncated in a way that changes parser semantics, whether a requested structured-output language is empty or ambiguous, and whether required prompt segments survive budget policies.

The non-goals are equally important. PromptABI does not prove that a model will resist prompt injection. It does not prove that a model will choose a safe tool. It does not evaluate factuality, alignment, toxicity, or behavioral compliance. It does not require model weights or generation. If an artifact contains arbitrary code, unsupported Jinja, provider behavior not captured by fixtures, or schemas outside the supported fragment, the correct behavior is to abstain or emit a bounded finding with explicit assumptions.

This narrowness makes the tool easier to trust. A structural counterexample does not depend on random seeds, temperature, model provider, or GPU hardware. If the artifact composition admits the unsafe state, that state exists.

20 Related Systems

PromptABI is adjacent to but distinct from model execution verifiers, prompt-injection scanners, schema validators, grammar-constrained decoding libraries, and provider SDK test suites. Model execution verifiers focus on tensors, devices, dtypes, shapes, checkpoint compatibility, or export gates. Prompt-injection scanners often attempt behavioral or semantic classification. Schema validators check a schema against an instance, but not necessarily the product of schema, tokenizer, grammar backend, stop policy, and application parser. Constrained-decoding libraries enforce output languages but may not report whether their language matches the downstream parser or whether stop logic truncates inside that language.

PromptABI's niche is the composition boundary. It verifies that tokenizer, template, tool, grammar, provider, and budget artifacts agree. It should integrate with existing libraries rather than replace them: validators provide ground truth for schema fragments, tokenizer libraries provide differential oracles, and provider fixtures define offline request/response semantics.

The closest analogy is a static analyzer for ABI compatibility. The concern is not whether either side of an interface is individually well-written; it is whether their composed contract remains valid as artifacts evolve.

21 Roadmap Without Step Enumeration

The roadmap is best understood as a progression from stable surfaces to formal checks to ecosystem integration. The skeleton, core artifact model, loader layer, diagnostic contract, CLI, tokenizer abstraction, finite automata layer, finite-state transducer layer, and finite-contract solver now provide stable surfaces. The next phase should implement source-span tracking, deterministic diagnostic snapshots, and the first role-boundary, stop-reachability, grammar-emptiness, and budget-survival checkers. Corpus growth, lockfiles, drift detection, SARIF, GitHub Actions, pre-commit hooks, HTML reports, suppression policies, and plugin APIs turn the verifier from a library into an everyday CI tool.

The research roadmap adds reproducibility packages, executable specifications, proof sketches for supported fragments, mutation-based fuzzing, cross-version CI, and real-bug benchmark suites. The community roadmap adds contribution infrastructure, compatibility matrices, integration guides, launch assets, and governance. These activities are not separate from the technical work. They are how a formal verifier becomes durable enough for production teams to trust and for researchers to reproduce.

22 Current Smoke Results

The current codebase was validated with focused tests for the package, artifact model, formal core, tokenizer abstraction, configuration loader, public API, and CLI workflow. The tokenizer tests exercise normalization, added tokens, special tokens, round-trip metadata, Hugging Face ByteLevel BPE behavior, `tiktoken cl100k_base`, and a trained SentencePiece model. The CLI tests exercise config discovery from nested directories, artifact-location overrides, deterministic JSON output, SARIF output, missing-artifact failures, cache directory preparation, quiet/verbose text controls, and configurable exit-code thresholds. The formal tests exercise automata intersection, difference, emptiness, shortest witnesses, transducer composition, projection, over-approximation, finite-domain counterexample extraction, minimal unsat cores, and the installed Z3 backend. The CLI was also run against the minimal typed-artifact example configuration, producing a passing informational diagnostic. The smoke benchmark runs the skeleton verifier repeatedly on the example config and reports a deterministic JSON result. These are small but important baselines: they prove that the repository can already execute code, not just describe a future tool.

Line	**Current status**
Focused tests	Artifact model, public API, config loading, and CLI workflow behavior pass in isolation.
Formal core tests	Automata reachability and difference, transducer composition/projection/over-approximation, finite-domain solving, unsat-core extraction, and Z3-backed assignment extraction pass against executable code.
Tokenizer differential tests	Byte-level normalization and added-token behavior, Hugging Face ByteLevel BPE, `tiktoken`, and SentencePiece adapters match real backend behavior.
CLI smoke	`promptabi verify -config examples/minimal/promptabi.json` returns pass, and `-fail-on`, `-artifact`, `-cache-dir`, `-quiet`, and `-verbose` are covered.
Benchmark smoke	Repeated config load and verification completes on CPU with JSON output.
Docs smoke	MkDocs-ready navigation and pages exist for the initial structure.

These are not the final experiments. They are the baseline benchlines that later experiments will extend: every new checker should add focused tests, fixture cases, corpus cases where relevant, and a benchmark line that can be compared across revisions.

23 Why CPU-Only Is a Strength

Many LLM quality questions are inherently behavioral and require model access. PromptABI intentionally avoids those claims. Tokenization runs on CPU. Chat-template rendering runs on CPU. JSON Schema normalization, grammar compilation, automata and transducer operations, parser checks, source-span tracking, fixture replay, and truncation simulation run on CPU. Provider compatibility can be tested from recorded fixtures. Context-window analysis depends on token counts and framework policies, not logits.

This makes PromptABI deployable in environments where GPU inference, external API calls, and prompt upload are not acceptable. A company can run the verifier inside CI on private prompt artifacts. A maintainer can run corpus checks on a laptop. A paper artifact can be reproduced without model weights. A pull request can fail because a tokenizer config changed, not because a stochastic generation happened to expose the bug once.

The CPU-only boundary also sharpens the formal claims. The tool can say "this delimiter is forgeable" or "this stop sequence can truncate valid JSON" with a structural witness. It should not say "the model is safe." That honesty is a competitive advantage.

24 Conclusion

PromptABI is positioned to verify a layer of LLM systems that is both fragile and under-tooled: the discrete interface between application data structures, prompt rendering, tokenization, constrained output, provider protocols, and application parsing. The first repository milestone establishes the engineering foundation: typed package, artifact model, tokenizer abstraction, CLI, public API, diagnostics, tests, docs, examples, fixtures, benchmarks, and contribution guidance. The research foundation is the transducer view of prompts, tokenizers, grammars, stop policies, parsers, and budget policies.

The next milestones should add source spans, stable diagnostic snapshots, and then the first formal checkers without changing the public surface. If the project maintains deterministic outputs, minimized witnesses, clear abstention boundaries, and real corpus validation, it can become both a practical CI tool and a credible systems research contribution. The core insight is simple but powerful: many production LLM failures happen before the neural network is involved, and those failures can be found before deployment.